TUGAS BESAR

Teori Bahasa Formal dan Otomata

13524007 Muhammad Ashkar
13524039 Rhenaldy Cahyadi Putra
13524069 Miguel Rangga Deardo Sinaga
13524097 Muhammad Faiz Alfada Dharma

DAFTAR ISI

BAB I TEORI SINGKAT **3**
1.1 Compiler 3
1.2 Lexical Analysis 3
1.3 Token 4
1.4 Finite Automata dan DFA 5
BAB II PERANCANGAN DFA **6**
2.1 Gambaran Umum 6
2.2 Pendekatan Perancangan DFA 7
2.3 Desain & Diagram DFA 8
2.3.1 DFA-1: Identifier dan Keyword 11
2.3.2 DFA-2: Number (Minus, Integer, dan Real) 13
2.3.3 DFA-3: String dan Char 14
2.3.4 DFA-4a: Comment {...} 15
2.3.5 DFA-4b: Comment (*...*) 16
2.3.6 DFA-5: Operator 17
2.3.7 DFA-6: Single Character Symbols 18
BAB III IMPLEMENTASI **19**
3.1 Struktur Program 19
3.2 Perancangan Program 20
3.2.1 Main Program 20
3.2.2 Reader 20
3.2.3 Lexer 20
3.2.4 Alur Kerja Program 21
BAB IV PENGUJIAN **21**
BAB V KESIMPULAN DAN SARAN **37**
5.1 Kesimpulan 37
LAMPIRAN **38**
REFERENSI **38**

BAB I TEORI SINGKAT

1.1 Compiler

Compiler merupakan sebuah program yang berfungsi untuk menerjemahkan source code yang ditulis dalam bahasa pemrograman tingkat tinggi menjadi kode mesin atau bentuk lain yang dapat dijalankan oleh komputer. Proses ini dilakukan melalui beberapa tahapan yang terstruktur agar kode yang dihasilkan sesuai dengan aturan bahasa yang digunakan.

Secara umum, proses kompilasi terdiri dari beberapa tahap utama, yaitu:

- Lexical Analysis, yaitu proses mengubah rangkaian karakter dalam source code menjadi token.
- Syntax Analysis, yaitu proses memeriksa apakah susunan token sesuai dengan aturan tata bahasa (grammar).
- Semantic Analysis, yaitu proses memastikan makna dari program sudah benar, seperti kesesuaian tipe data dan deklarasi variabel.
- Intermediate Code Generation, yaitu proses menghasilkan representasi perantara dari program yang dapat digunakan untuk tahap selanjutnya.

Pada milestone ini, fokus utama berada pada tahap Lexical Analysis, yaitu tahap awal dalam proses kompilasi yang bertugas untuk menghasilkan token dari source code. Tahap ini menjadi dasar bagi tahap-tahap selanjutnya, sehingga keberhasilan analisis leksikal sangat mempengaruhi proses kompilasi secara keseluruhan.

1.2 Lexical Analysis

Lexical Analysis atau analisis leksikal merupakan tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, source code yang berupa rangkaian karakter diproses untuk menghasilkan sekumpulan token yang memiliki makna sesuai dengan aturan bahasa Arion.

Proses analisis leksikal dilakukan oleh komponen yang disebut *lexer* atau *lexical analyzer*. Lexer membaca input secara berurutan dari kiri ke kanan, satu karakter demi

satu karakter, kemudian mengelompokkan karakter-karakter tersebut menjadi token berdasarkan pola tertentu.

Dalam prosesnya, lexer memiliki beberapa fungsi utama, yaitu:

- Mengidentifikasi dan mengelompokkan karakter menjadi token sesuai dengan spesifikasi bahasa.
- Mengabaikan elemen yang tidak berpengaruh terhadap struktur program, seperti spasi, tab, dan komentar.
- Mendeteksi kesalahan leksikal, yaitu ketika ditemukan karakter atau pola yang tidak dikenali oleh aturan bahasa.

Lexer Arion bekerja dengan prinsip *longest match*, yaitu membaca karakter secara berurutan dan membentuk token sepanjang mungkin selama masih sesuai dengan pola suatu token yang valid. Proses ini akan berhenti ketika karakter berikutnya tidak lagi memenuhi pola tersebut, sehingga token yang terbentuk merupakan rangkaian karakter terpanjang yang masih valid.

Sebagai contoh, untuk potongan kode berikut if123, lexer akan menghasilkan ident(if123), bukan ifsy intcon(123), karena seluruh rangkaian karakter tersebut masih memenuhi aturan pembentukan identifier.

Sebagai ilustrasi lain, untuk potongan kode berikut:

```
if a > 10 then
```

lexer akan menghasilkan token:

```
ifsy
ident(a)
gtr
intcon(10)
thensy
```

Proses identifikasi token pada lexer dapat diimplementasikan menggunakan Deterministic Finite Automata (DFA). Dengan menggunakan DFA, pembacaan karakter dapat dilakukan secara sistematis melalui perpindahan state, sehingga setiap urutan karakter dapat diklasifikasikan menjadi token yang sesuai.

1.3 Token

Token merupakan unit terkecil yang memiliki makna dalam suatu bahasa pemrograman. Pada tahap analisis leksikal (lexical analysis), source code yang awalnya berupa

rangkaian karakter akan dipecah menjadi sekumpulan token yang lebih terstruktur dan mudah diproses oleh tahap selanjutnya dalam proses kompilasi.

Setiap token umumnya terdiri dari dua komponen utama, yaitu:

- **Tipe token (token type)**, yang menunjukkan kategori token, seperti *ident, intcon, realcon, charcon,* dan *string,*
- **Nilai token (token value)**, yang merupakan representasi konkret dari token tersebut dalam source code.

Selain token, dikenal pula istilah lexeme, yaitu rangkaian karakter dalam source code yang membentuk suatu token. Lexeme merupakan representasi konkret dari token yang ditemukan pada proses analisis leksikal. Sebagai contoh, pada potongan kode if a > 10, lexeme yang terbentuk adalah if, a, >, dan 10, yang masing-masing dipetakan menjadi token ifsy(), ident(a), gtr, dan intcon(10).

Dalam bahasa pemrograman Arion, lexer mampu mengenali berbagai token yang telah ditentukan dalam spesifikasi, seperti *ident*, *intcon*, *realcon*, *charcon*, dan *string*, serta berbagai operator, keyword, dan simbol khusus lainnya. Dengan mengubah source code menjadi token-token, proses analisis sintaks (parsing) dapat dilakukan dengan lebih mudah karena struktur program sudah direpresentasikan dalam bentuk unit-unit yang bermakna.

1.4 Finite Automata dan DFA

Finite Automata (FA) merupakan model komputasi matematis yang digunakan untuk mengenali pola dalam suatu rangkaian karakter. Secara formal, sebuah Finite Automata dapat didefinisikan sebagai sebuah 5-tuple:

$$M=(Q,\Sigma,\delta,q0,F)$$

dengan:

- Q adalah himpunan state (keadaan).
- Σ adalah himpunan simbol input (alfabet).
- δ adalah fungsi transisi, yaitu $\delta : Q \times \Sigma \rightarrow Q$.
- $q0 \in Q$ adalah state awal.
- $F \subseteq Q$ adalah himpunan state akhir (*accepting states*).

Finite Automata bekerja dengan membaca simbol input secara berurutan dan berpindah dari satu state ke state lainnya sesuai dengan fungsi transisi yang didefinisikan. Suatu input dikatakan diterima apabila proses pembacaan berakhir pada salah satu state dalam himpunan F.

Salah satu jenis Finite Automata yang digunakan dalam implementasi lexical analyzer adalah Deterministic Finite Automata (DFA). DFA memiliki sifat deterministik, yaitu untuk setiap pasangan state dan simbol input, hanya terdapat satu kemungkinan state tujuan. Dengan kata lain, fungsi transisi pada DFA bersifat tunggal dan tidak ambigu.

Dalam konteks lexical analysis, DFA digunakan untuk mengenali pola token dengan membaca karakter secara berurutan. Setiap jenis token direpresentasikan sebagai pola tertentu dalam DFA, dan suatu token dianggap valid apabila proses pembacaan berakhir pada state akhir yang sesuai.

Sebagai contoh, DFA untuk mengenali identifier dirancang dengan aturan sebagai berikut:

- Dari state awal, jika membaca huruf, maka berpindah ke state identifier.
- Pada state identifier, selama membaca huruf atau digit, tetap berada pada state tersebut.
- Jika pembacaan berhenti pada state identifier, maka token yang terbentuk merupakan identifier yang valid.

BAB II PERANCANGAN DFA

2.1 Gambaran Umum

Pada tahap ini dilakukan perancangan Deterministic Finite Automata (DFA) yang akan digunakan untuk mengenali token dalam bahasa Arion. DFA dirancang untuk membaca karakter input secara berurutan dan mengelompokkan karakter tersebut menjadi lexeme yang valid berdasarkan pola tertentu. Setiap lexeme yang dikenali kemudian diklasifikasikan menjadi token.

Struktur Umum Sistem DFA

Secara umum, proses pengenalan token menggunakan DFA dalam sistem ini mengikuti alur berikut:

1. Membaca karakter dari input satu per satu
2. Menentukan state awal berdasarkan karakter pertama
3. Melakukan transisi antar state sesuai dengan karakter berikutnya
4. Menghentikan pembacaan ketika tidak terdapat transisi yang valid
5. Menghasilkan lexeme
6. Mengklasifikasikan lexeme menjadi token yang sesuai

2.2 Pendekatan Perancangan DFA

Dalam perancangan lexical analyzer kami, terdapat dua pendekatan yang kami gunakan dalam membangun DFA untuk mengenali token, yaitu pendekatan eksplisit berbasis token dan pendekatan berbasis pola lexeme.

2.2.1 Pendekatan DFA Eksplisit Berbasis Token

Pada pendekatan ini, setiap token direpresentasikan secara langsung sebagai lintasan dalam DFA hingga mencapai state final yang unik. Sebagai contoh, keyword while direpresentasikan sebagai transisi berurutan, yaitu -> q0 (w) -> q1 (h) -> q2 (i) -> q3 (l) -> q4 (e) -> q5 (whilesy).

Pendekatan ini menghasilkan jumlah state yang sangat besar karena setiap token harus diuraikan karakter demi karakter. Kelebihan pendekatan ini adalah klasifikasi token dilakukan langsung melalui state final, sehingga tidak memerlukan proses tambahan setelah DFA selesai. Namun, karena kompleksitas DFA yang tinggi, implementasi dengan pendekatan ini menjadi sulit diimplementasikan dan dipelihara.

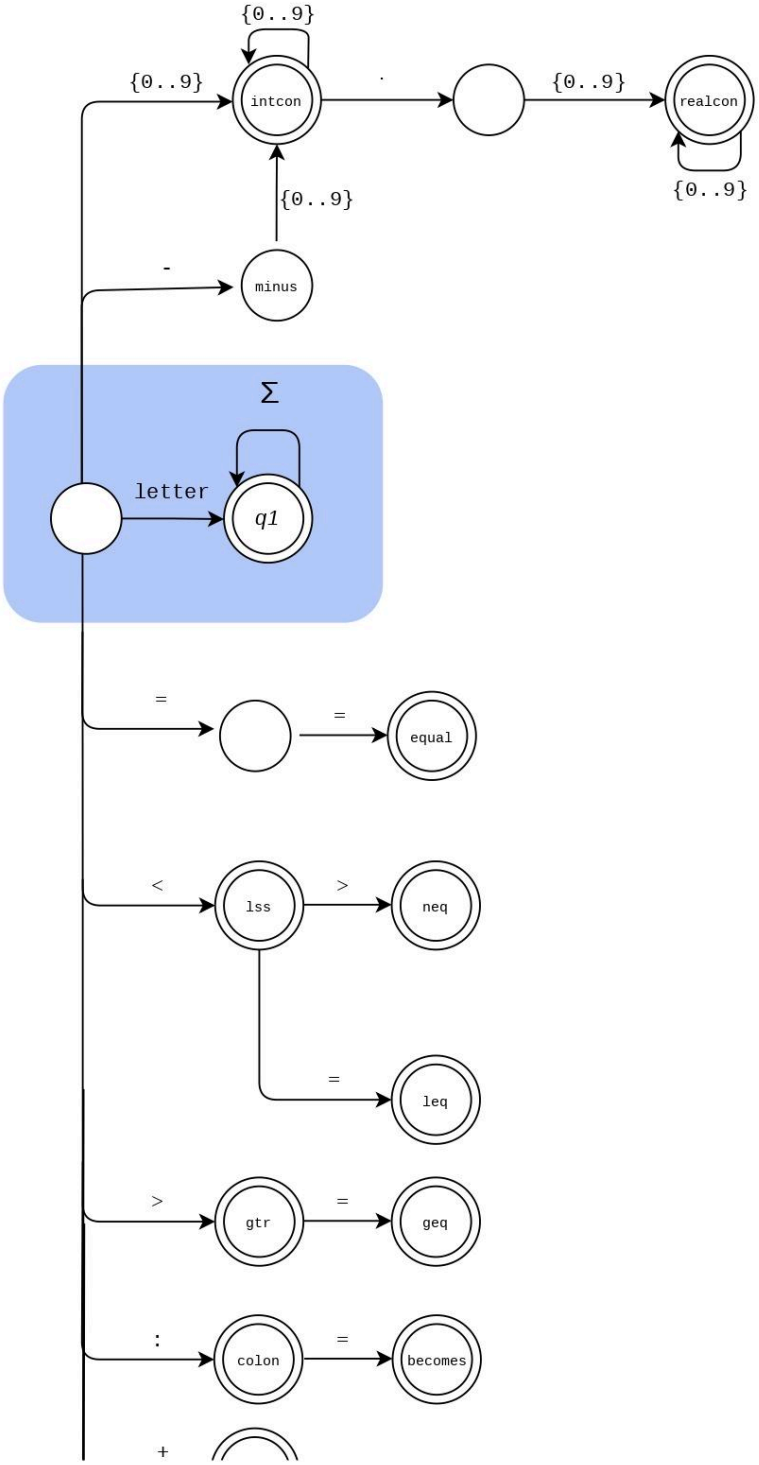
Dalam tugas ini, pendekatan eksplisit berbasis token **digunakan pada DFA-2 hingga DFA-6**, yaitu untuk token number, string & char, comment, operator, dan single character simbol. Hal ini dikarenakan jumlah pola token pada kategori tersebut relatif terbatas dan memiliki struktur yang spesifik, sehingga masih dapat direpresentasikan secara langsung tanpa menyebabkan jumlah state yang terlalu besar.

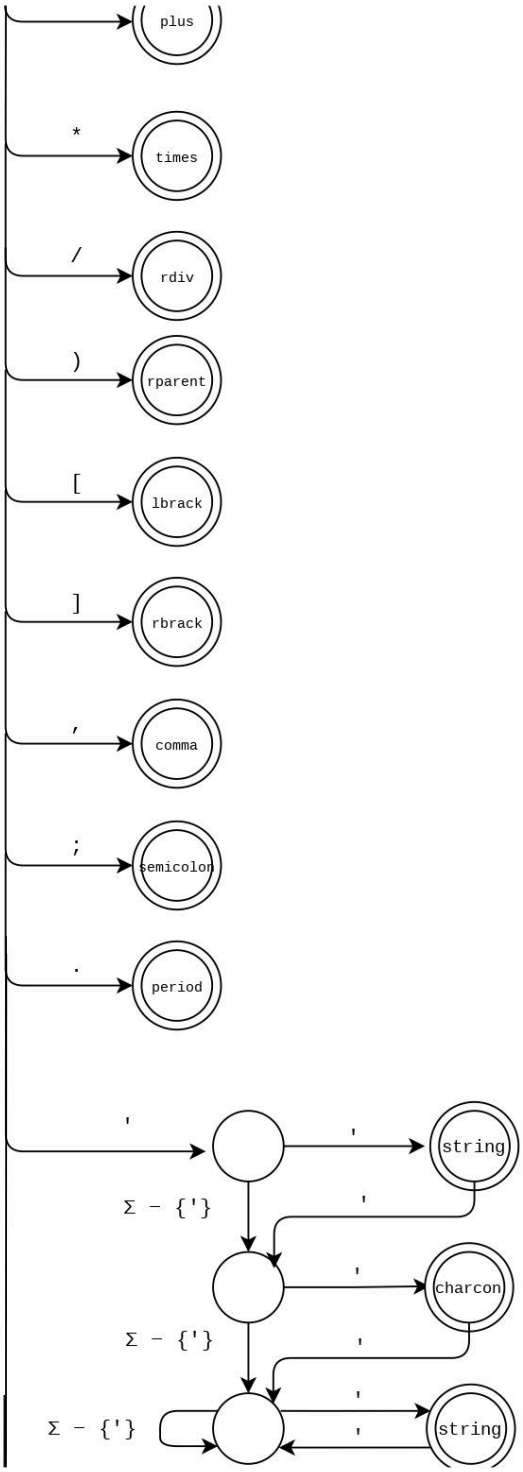
2.2.2 Pendekatan DFA Berbasis Lexeme

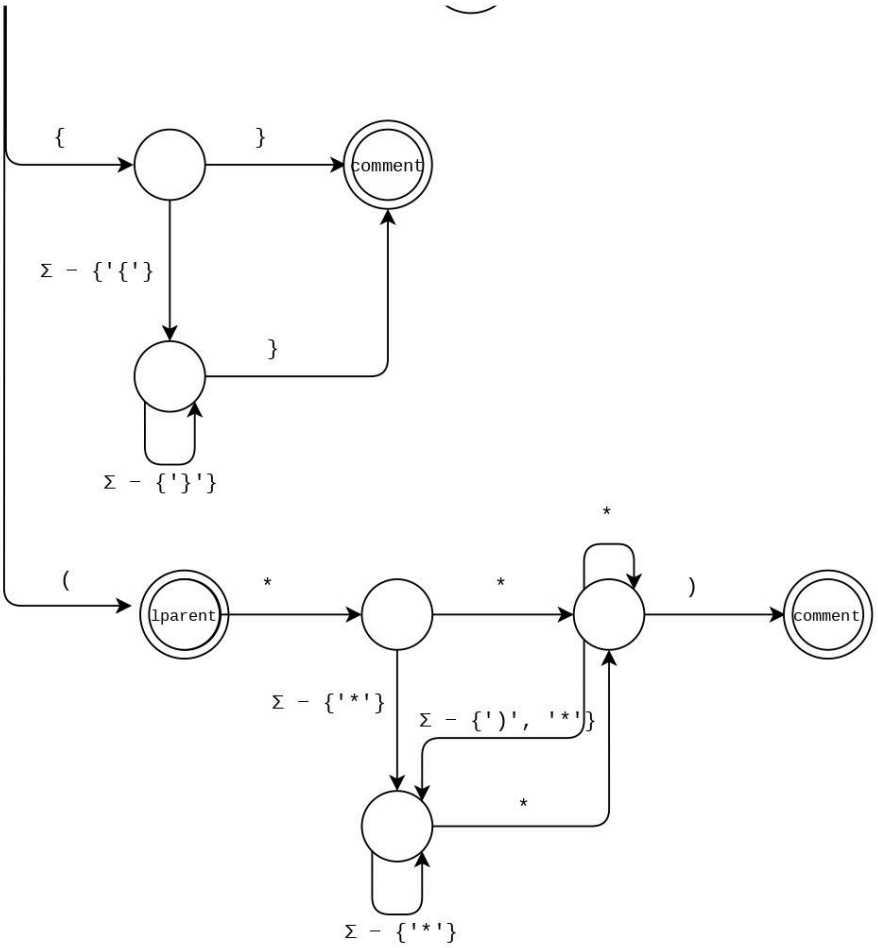
Pendekatan kedua adalah dengan mengenali token berdasarkan pola umum (pattern) menggunakan DFA, kemudian melakukan klasifikasi lanjutan terhadap lexeme yang dihasilkan. Sebagai contoh, identifier dikenali menggunakan pola: letter (letter | digit)*. DFA hanya bertugas mengenali pola tersebut hingga mencapai state final, tanpa membedakan apakah lexeme tersebut merupakan identifier atau keyword. Setelah lexeme terbentuk, dilakukan pengecekan terhadap tabel keyword (keyword table) untuk menentukan jenis token yang sesuai.

Pendekatan ini sesuai dengan konsep yang dijelaskan dalam literatur yang tertera pada referensi, di mana token dikenali berdasarkan pola yang didefinisikan menggunakan *regular expression*. Pendekatan ini menghasilkan DFA dengan jumlah state yang lebih sedikit, lebih modular, dan lebih mudah diimplementasikan dibandingkan pendekatan eksplisit. Dalam tugas ini, pendekatan berbasis pola lexeme **digunakan pada DFA-1,** yaitu untuk mengenali identifier dan keyword. Hal ini dikarenakan jumlah kemungkinan lexeme pada kategori ini sangat banyak (tidak terbatas), sehingga tidak efisien jika direpresentasikan secara eksplisit dalam DFA berbasis token.

2.3 Desain & Diagram DFA




plus
*
times
/
rdiv
)
rparent
[
lbrack
]
rbrack
,
comma
;
semicolon
.
period
'
'
string
Σ − {'}
'
'
charcon
Σ − {'}
'
'
Σ − {'}
'
string



Gambar 1, DFA keseluruhan program yang diimplementasikan

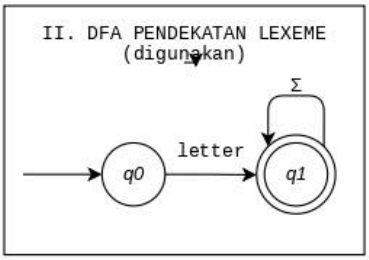
2.3.1 DFA-1: Identifier dan Keyword

DFA ini digunakan untuk mengenali pola identifier yang terdiri dari huruf diikuti oleh kombinasi huruf dan digit. Proses dimulai dari state awal ketika membaca karakter huruf, kemudian bertransisi ke state q1 yang akan terus menerima karakter huruf maupun digit.

DFA ini tidak secara langsung membedakan antara identifier dan keyword. Setelah lexeme dikenali, dilakukan proses klasifikasi lanjutan dengan membandingkan lexeme terhadap tabel keyword. Jika lexeme ditemukan dalam tabel, maka diklasifikasikan sebagai keyword, jika tidak maka sebagai identifier.

DFA untuk identifier dan keyword didefinisikan sebagai $M_1 = (Q_1, \Sigma_1, \delta_1, q_0, F_1)$ dengan:

- Q_1 = {q0, q1}
- Σ_1 = {letter, digit}
- q_0 = q0 (state awal)
- F_1 = {q1} (state final)

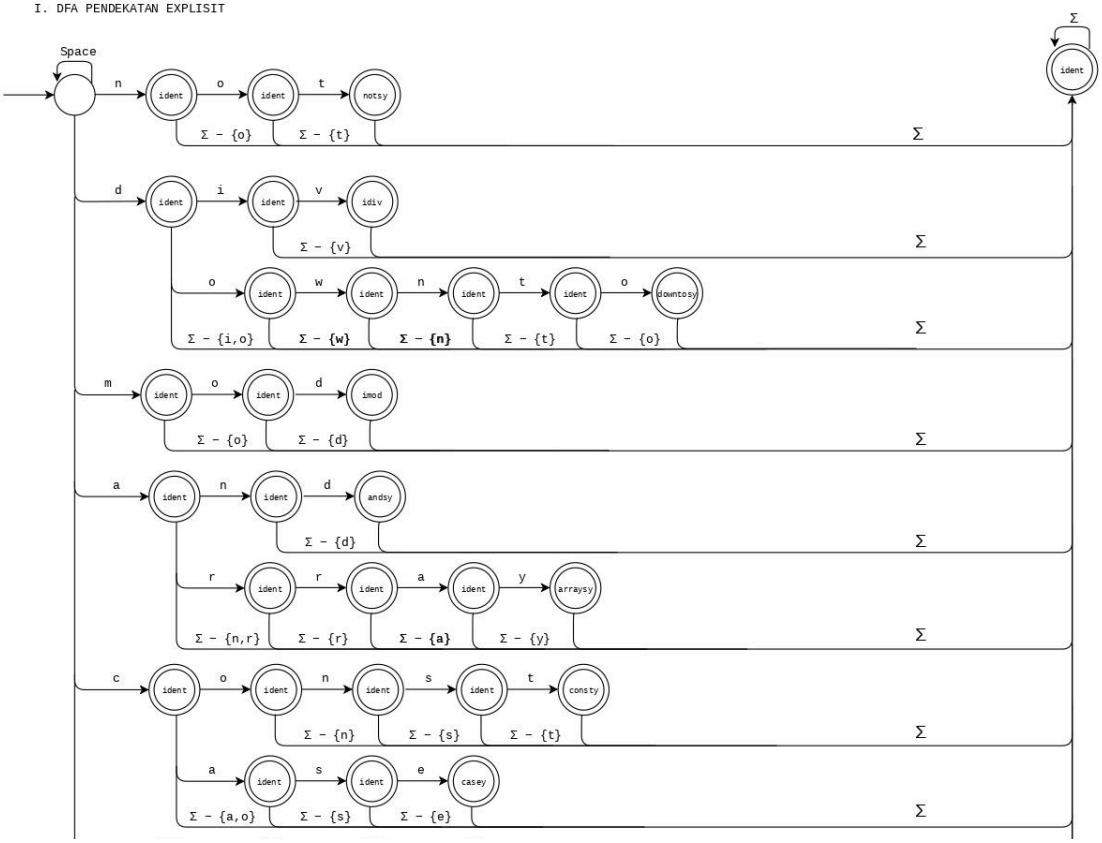


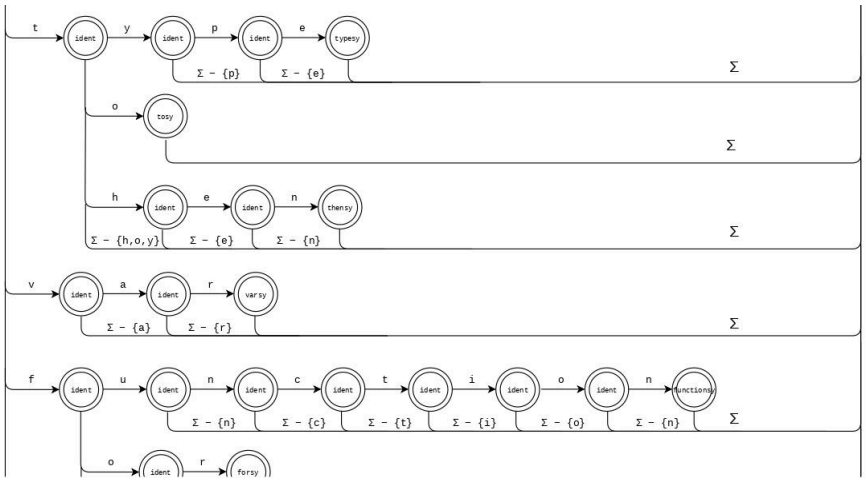
Gambar 2, DFA-1: Identifier dan Keyword (pendekatan Lexeme, diimplementasikan). Simbol Σ pada gambar merujuk pada Alfabet DFA-1 {letter, digit}.

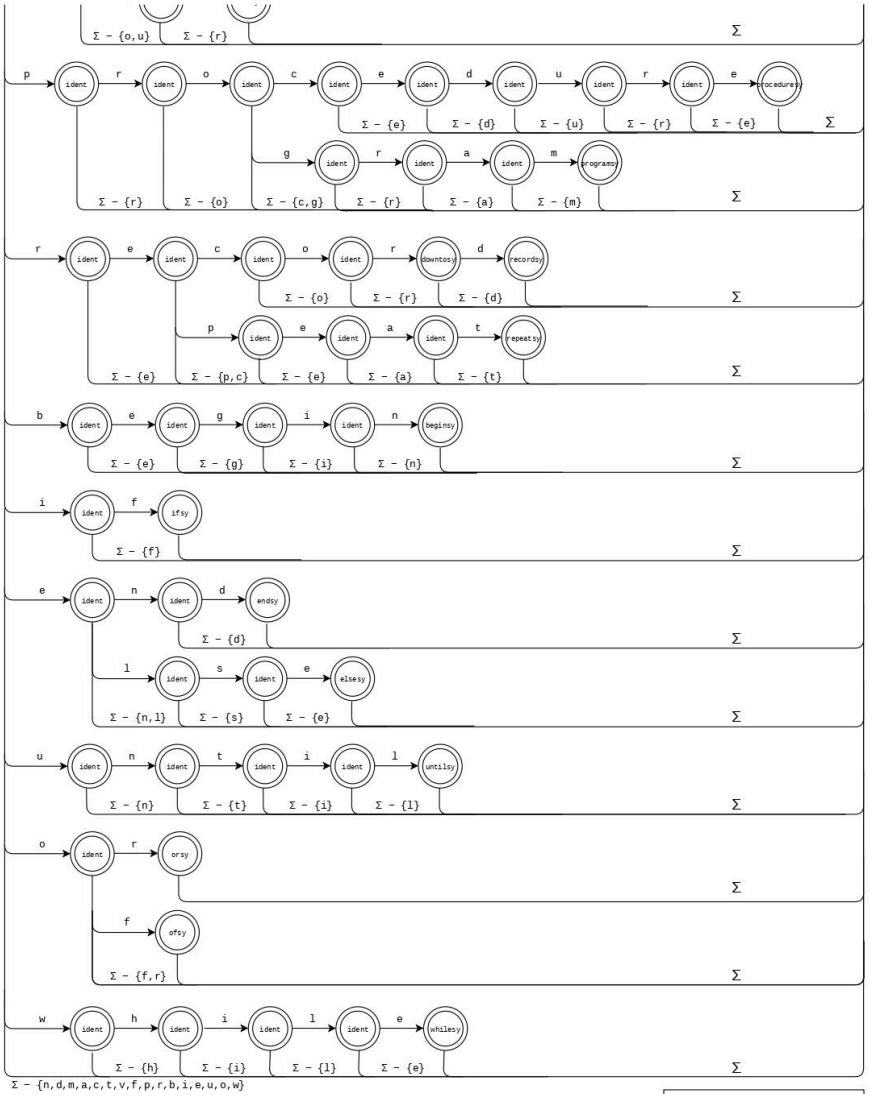
δ_1	letter[a-z][A-Z]	digit[0..9]
-> q0	q1	–
q1*	q1	q1

Adapun di bawah ini adalah desain DFA-1 yang menggunakan pendekatan eksplisit. Dapat diperhatikan bahwa jumlah state menjadi sangat banyak.

I. DFA PENDEKATAN EXPLISIT







Gambar 3, DFA-1: Identifier dan Keyword (pendekatan eksplisit per token, tidak diimplementasikan). Simbol Σ pada gambar merujuk pada Alfabet DFA-1 {letter, digit}.

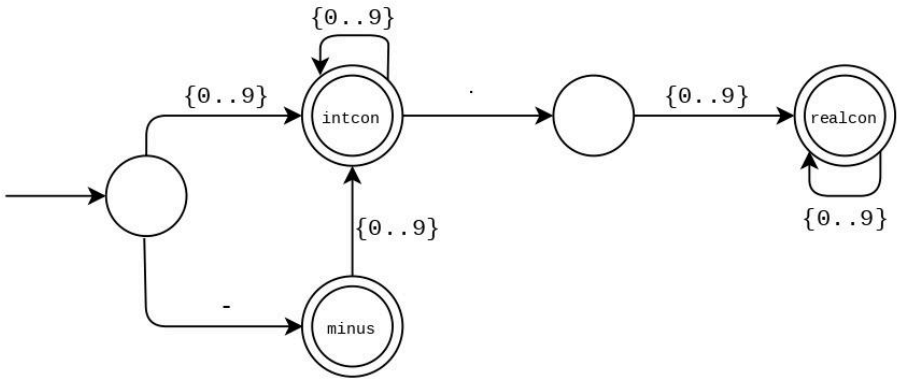
2.3.2 DFA-2: Number (Minus, Integer, dan Real)

DFA ini digunakan untuk mengenali konstanta numerik berupa bilangan integer dan real. Apabila setelah digit ditemukan tanda titik (.), DFA akan berpindah ke state antara qdot yang menandakan bahwa bilangan tersebut berpotensi menjadi bilangan real. Selanjutnya, jika setelah tanda titik ditemukan digit, maka DFA berpindah ke state realcon dan terus menerima digit sebagai bagian dari bilangan real.

Selain itu, DFA juga mendukung bilangan negatif dengan membaca tanda minus (-) pada awal input. Setelah berada pada state minus, DFA akan mengharapkan digit untuk melanjutkan pembentukan bilangan integer.

DFA untuk number didefinisikan sebagai $\mathbf{M_2 = (Q_2, \Sigma_2, \delta_2, q_0, F_2)}$ dengan:

- Q_2 = {q0, intcon, qdot, realcon, minus}
- Σ_2 = {digit, '.', '-'}
- q_0 = q0 (state awal)
- F_2 = {intcon, realcon, minus} (state final)



Gambar 4, DFA-2: Number

δ_2	**digit**	**'.'**	**'-'**
-> q0	intcon	–	minus
intcon*	intcon	qdot	–
qdot	realcon	–	–
realcon*	realcon	–	–
minus*	intcon	–	–

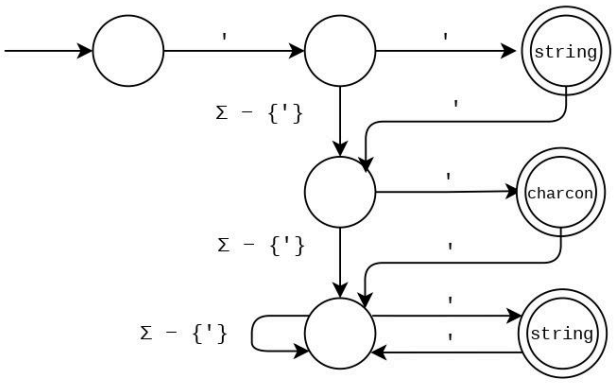
2.3.3 DFA-3: String dan Char

DFA ini digunakan untuk mengenali literal string dan karakter yang diapit oleh tanda kutip tunggal ('). Proses dimulai dari state awal saat membaca tanda kutip pembuka, kemudian DFA akan membaca karakter di dalamnya. Jika setelah tanda kutip pembuka langsung ditemukan tanda kutip penutup, maka DFA berpindah ke state string, yang merepresentasikan string kosong. Jika ditemukan satu karakter diikuti tanda kutip penutup, maka DFA berpindah ke state charcon yang merepresentasikan karakter

tunggal. Perbedaan antara state $string_1$ dan $string_2$ muncul karena adanya mekanisme escape menggunakan tanda kutip tunggal, di mana dua tanda kutip berturut-turut ('') merepresentasikan satu karakter kutip dalam string atau karakter.

DFA untuk string dan character didefinisikan sebagai $\mathbf{M_3 = (Q_3, \Sigma_3, \delta_3, q_0, F_3)}$ dengan:

- Q_3 = {q0, q1, q2, q3, $string_1$, $string_2$, charcon}
- Σ_3 = {' , other}
- q_0 = q0 (state awal)
- F_3 = {$string_1$, $string_2$, charcon} (state final)



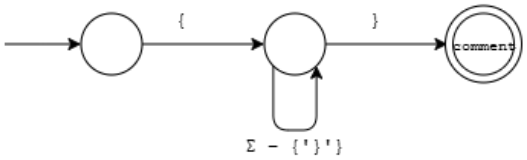
Gambar 5, DFA-3: String dan Char

δ_3	'	other
-> q0	q1	–
q1	$string_1$*	q2
q2	charcon*	q3
q3	$string_2$*	q3
$string_1$*	q2	–
$string_2$*	q3	–
charcon*	q3	–

2.3.4 DFA-4a: Comment {...}

DFA ini digunakan untuk mengenali komentar yang diawali dengan tanda kurung kurawal buka ({) dan diakhiri dengan tanda kurung kurawal tutup (}). Proses dimulai dari

state awal ketika membaca simbol {, kemudian DFA berpindah ke state q1 yang merepresentasikan bagian dalam komentar. Pada state ini, DFA akan terus menerima semua karakter hingga menemukan simbol } sebagai penutup komentar.

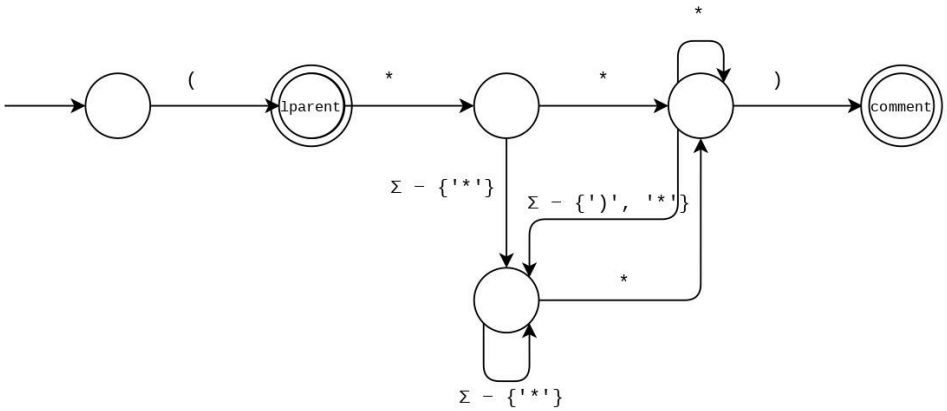


Gambar 6, DFA-4a: Comment {...}

δ_4a	{	}	other
-> q0	q1	—	—
q1	—	comment*	q1
comment*	—	—	—

2.3.5 DFA-4b: Comment (*...*)

DFA ini digunakan untuk mengenali komentar yang diawali dengan pasangan simbol (* dan diakhiri dengan *). Proses dimulai dari state awal saat membaca simbol (yang terlebih dahulu dikenali sebagai token lparent. Jika setelah itu ditemukan simbol *, maka DFA melanjutkan ke proses pengenalan komentar. Hal ini memungkinkan DFA membedakan antara simbol (sebagai token tunggal dan sebagai awal komentar.



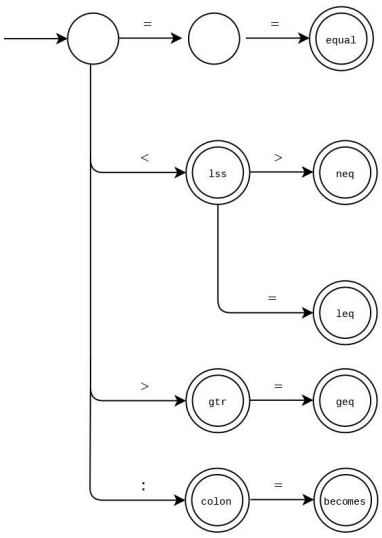
Gambar 7, DFA-4b: Comment (...*)*

δ_4b	(	*	)	other
-> q0	lparent*	–	–	–
lparent*	–	q1	–	–
q1	–	q3	–	q2
q2	–	q3		q2
q3	–	q3	comment*	q2
comment*	–	–	–	–

2.3.6 DFA-5: Operator

DFA ini digunakan untuk mengenali operator yang terdiri dari satu atau dua karakter, yaitu <, >, :, dan = beserta variasinya seperti <=, <>, >=, :=, dan ==. Proses dimulai dari state awal ketika membaca salah satu karakter tersebut, kemudian DFA akan berpindah ke state final sementara seperti lss, gtr, atau colon. Setelah itu, DFA akan membaca karakter berikutnya untuk menentukan apakah operator tersebut merupakan operator satu karakter atau dua karakter.

Jika ditemukan karakter lanjutan yang valid, DFA akan berpindah ke state final yang sesuai seperti leq, neq, geq, becomes, atau equal. Jika tidak ditemukan karakter lanjutan yang sesuai, maka operator satu karakter langsung diterima sebagai token.



Gambar 8, DFA-5: Operator

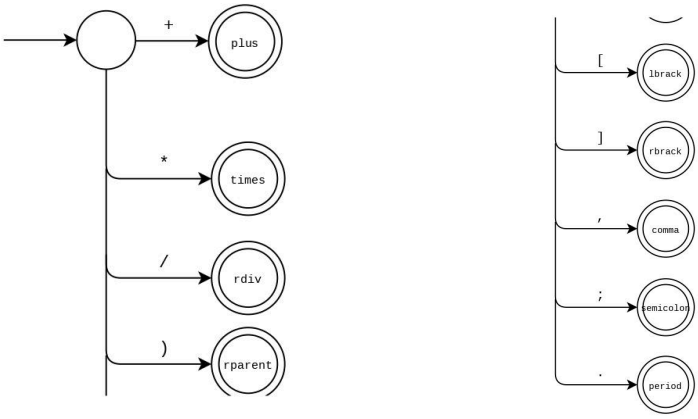
δ_5	<	>	:	=
-> q0	lss*	gtr*	colon*	q1
q1	–	–	–	equal*
lss*	–	neq*	–	leq*
gtr*	–	–	–	geq*
colon*	–	–	–	becomes*
equal*	–	–	–	–
neq*	–	–	–	–
leq*	–	–	–	–
geq*	–	–	–	–
becomes*	–	–	–	–

2.3.7 DFA-6: Single Character Symbols

DFA ini digunakan untuk mengenali simbol yang terdiri dari satu karakter, seperti +, -, *, /,), [,], ,, ;, dan .. Proses dimulai dari state awal ketika membaca salah satu simbol tersebut, dan DFA langsung berpindah ke state final yang merepresentasikan token yang sesuai tanpa memerlukan transisi lanjutan. Karena setiap simbol hanya terdiri dari satu karakter dan tidak memiliki kemungkinan perluasan menjadi token lain, DFA ini memiliki struktur yang sederhana dan tidak memerlukan state antara.

DFA untuk single character symbol didefinisikan sebagai $\mathbf{M_6 = (Q_6, \Sigma_6, \delta_6, q_0, F_6)}$ dengan:

- Q_6 = {q0, plus, minus, times, rdiv, rparent, lbrack, rbrack, comma, semicolon, period}
- Σ_6 = {+, -, *, /,), [,], ,, ;, .}
- q_0 = q0 (state awal)
- F_6 = {plus, minus, times, rdiv, rparent, lbrack, rbrack, comma, semicolon, period} (state final).



Gambar 9, DFA-6: Single Character Symbols

δ_6	+	-	*	/	)	[	]	,	;	.
-> q0	plus*	minus*	times*	rdiv*	rparent*	lbrack*	rbrack*	comma*	semicolon*	period*

BAB III IMPLEMENTASI

3.1 Struktur Program

```
JKW-Tubes-IF2224-2026/
├── README.md
├── doc/
│   └── Laporan-1-JKW.pdf
├── src/
│   ├── main.cpp
│   └── lexer/
│       ├── lexer.cpp
│       ├── lexer.hpp
│       ├── token.hpp
│       ├── reader.cpp
│       └── reader.hpp
└── test/
    ├── milestone-1/
    ├── milestone-2/
    ├── milestone-3/
    └── milestone-4/
```

3.2 Perancangan Program

Program terdiri dari tiga komponen utama, yaitu main program, Reader, dan Lexer. Ketiga komponen ini bekerja secara berurutan mulai dari menerima input path file, membuka dan membaca isi file, sampai mengubah karakter-karakter yang dibaca menjadi token.

3.2.1 Main Program

Bagian utama program berawal dengan meminta input berupa path dari pengguna, kemudian menggunakan modul reader untuk membuka file tersebut. Setelah file berhasil dibuka, program akan menginstansiasi objek lexer yang akan digunakan untuk proses selanjutnya. Program akan memanggil fungsi getNextToken() yang akan mengembalikan type token dan isi dari token tersebut ke main, dan akan langsung di ditulis ke file output. Dengan ini, main program hanya bertugas menghubungkan Reader dengan Lexer dan menulis ke file output tanpa ikut mengerjakan detail tokenisasi.

3.2.2 Reader

Reader berfungsi sebagai abstraksi proses pembacaan file secara karakter per karakter. Modul ini menyediakan fungsi open(path) untuk membuka file, get() untuk mengakses karakter saat ini, next() untuk berpindah ke karakter berikutnya, serta isEOF() untuk mendeteksi akhir file.

Pemisahan Reader dari Lexer memungkinkan proses input dan proses analisis dipisahkan secara jelas. Dengan demikian, Lexer dapat difokuskan sepenuhnya pada implementasi DFA tanpa perlu menangani detail I/O, sehingga meningkatkan modularitas dan kemudahan pengujian.

3.2.3 Lexer

Perancangan modul Lexer dilakukan dengan berfokus pada alur implementasi analisis leksikal yang mengikuti model Deterministic Finite Automata (DFA). Proses dimulai dari pengecekan token yang dilakukan fungsi getNextToken() yang akan membaca karakter secara berurutan dari Reader ke fungsi processChar(char c), di mana setiap karakter akan diproses berdasarkan state saat ini yang pada awalnya berada di state START. Pada tahap ini, karakter pertama digunakan untuk menentukan jenis token yang sedang dibentuk, sehingga lexer akan berpindah ke state yang sesuai.
Selanjutnya, lexer akan membangun lexeme dengan terus menambahkan karakter selama masih sesuai dengan pola state tersebut sehingga terbentuk rangkaian karakter terpanjang yang valid sesuai prinsip longest match. Ketika karakter berikutnya tidak lagi memenuhi pola, proses pembentukan lexeme dihentikan dan dilakukan klasifikasi token.

Setelah token dihasilkan, lexer akan kembali ke state awal untuk memproses karakter berikutnya.

3.2.4 Alur Kerja Program

Secara keseluruhan, alur kerja program dimulai dari pembacaan input file oleh Reader, kemudian karakter dikirim secara berurutan ke Lexer untuk diproses menjadi token. Lexer akan membaca karakter, melakukan transisi state, membentuk lexeme, dan menghasilkan token hingga seluruh input selesai diproses. Token yang dihasilkan kemudian dituliskan ke file output oleh main program.

BAB IV PENGUJIAN

Masukkan (Input)

```
program PythonGacor;

var
  x: real;
  y: real;
  i: integer;

begin
  x := 5;
  y := -6.7;

  if ((NOT (x <> y) AND (x <> y)) OR (x < 0) OR (x == y)) then
    writeln('Rhenaldy Cahyadi - 13524113')
  else
  begin
    writeln('Spesifikasi kurang lengkap');
    writeln('Deadline harus diperpanjang');
    writeln('/n');
  end;

  for i := 1 downto -4 do
    x := x + 1;
end.
```

Keluaran (output)

```
PROGRAMSY()
IDENT(PythonGacor)
SEMICOLON()
VARSY()
IDENT(x)
COLON()
IDENT(real)
SEMICOLON()
```

```
IDENT(y)
COLON()
IDENT(real)
SEMICOLON()
IDENT(i)
COLON()
IDENT(integer)
SEMICOLON()
BEGINSY()
IDENT(x)
BECOMES()
INTCON(5)
SEMICOLON()
IDENT(y)
BECOMES()
MINUS()
REALCON(6.7)
SEMICOLON()
IFSY()
LPARENT()
LPARENT()
NOTSY()
LPARENT()
IDENT(x)
NEQ()
IDENT(y)
RPARENT()
ANDSY()
LPARENT()
IDENT(x)
NEQ()
IDENT(y)
RPARENT()
RPARENT()
ORSY()
LPARENT()
IDENT(x)
LSS()
INTCON(0)
RPARENT()
ORSY()
LPARENT()
IDENT(x)
EQL()
IDENT(y)
RPARENT()
RPARENT()
THENSY()
IDENT(writeln)
LPARENT()
STRING('Rhenaldy Cahyadi - 13524113')
RPARENT()
ELSESY()
BEGINSY()
IDENT(writeln)
LPARENT()
```

```
STRING('Spesifikasi kurang lengkap')
RPARENT()
SEMICOLON()
IDENT(writeln)
LPARENT()
STRING('Deadline harus diperpanjang')
RPARENT()
SEMICOLON()
IDENT(writeln)
LPARENT()
STRING('/n')
RPARENT()
SEMICOLON()
ENDSY()
SEMICOLON()
FORSY()
IDENT(i)
BECOMES()
INTCON(1)
DOWNTOSY()
MINUS()
INTCON(4)
DOSY()
IDENT(x)
BECOMES()
IDENT(x)
PLUS()
INTCON(1)
SEMICOLON()
ENDSY()
PERIOD()
```

Masukkan (Input)

```
program TubesBasdatFauzanOtomata;

const
  rhen := 100;

type
  TubeRecord = record
    Deadline: real;
    Diperpanjang: string;
    please: string;
    JKW: string;
  end;

  TubeArray = array[10] of integer;

var
  data: TubeRecord;
  Tubes: TubeArray;
```

```
begin
  {
    Tolong spesifikasinya diperjelas lagi
  }
  (* Banyak banget perubahan di QNA *)

  data.Deadline := -23.59;
  data.please := 'please';
  data.Diperpanjang := data.please;
  data.JKW := 'Jok-';

  writeln('NOT AND OR div MOD');
  writeln('(* ini bukan comment *)', ' ', '{ ini komen kan }');
end.
```

Keluaran (output)

```
PROGRAMSY()
IDENT(TubesBasdatFauzanOtomata)
SEMICOLON()
CONSTSY()
IDENT(rhen)
BECOMES()
INTCON(100)
SEMICOLON()
TYPESY()
IDENT(TubeRecord)
UNKNOWN()
RECORDSY()
IDENT(Deadline)
COLON()
IDENT(real)
SEMICOLON()
IDENT(Diperpanjang)
COLON()
IDENT(string)
SEMICOLON()
IDENT(please)
COLON()
IDENT(string)
SEMICOLON()
IDENT(JKW)
COLON()
IDENT(string)
SEMICOLON()
ENDSY()
SEMICOLON()
IDENT(TubeArray)
UNKNOWN()
ARRAYSY()
LBRACK()
INTCON(10)
RBRACK()
OFSY()
IDENT(integer)
```

```
SEMICOLON()
VARSY()
IDENT(data)
COLON()
IDENT(TubeRecord)
SEMICOLON()
IDENT(Tubes)
COLON()
IDENT(TubeArray)
SEMICOLON()
BEGINSY()
COMMENT(
    Tolong spesifikasinya diperjelas lagi
  )
COMMENT( Banyak banget perubahan di QNA )
IDENT(data)
PERIOD()
IDENT(Deadline)
BECOMES()
MINUS()
REALCON(23.5)
SEMICOLON()
IDENT(data)
PERIOD()
IDENT(please)
BECOMES()
STRING('please')
SEMICOLON()
IDENT(data)
PERIOD()
IDENT(Diperpanjang)
BECOMES()
IDENT(data)
PERIOD()
IDENT(please)
SEMICOLON()
IDENT(data)
PERIOD()
IDENT(JKW)
BECOMES()
STRING('Jok-')
SEMICOLON()
IDENT(writeln)
LPARENT()
STRING('NOT AND OR div MOD')
RPARENT()
SEMICOLON()
IDENT(writeln)
LPARENT()
STRING('(* ini bukan comment *)')
COMMA()
CHARCON(' ')
COMMA()
STRING('{ ini komen kan }')
RPARENT()
SEMICOLON()
```

```
ENDSY()
PERIOD()
```

Masukkan (Input)

```
program NormalBangetLengkapProgramnya;

type
  AngkaArray = array[5] of integer;

  DataLoop = record
    i: integer;
    j: integer;
    status: string;
  end;

var
  data: DataLoop;
  arr: AngkaArray;
  hasil: integer;

procedure TulisPesan(teks: string);
begin
  writeln(teks);
end;

function Hitung(x: integer): integer;
begin
  Hitung := (x div 3) MOD 2;
end;

begin
  data.i := 0;
  data.j := 0;
  data.status := 'awal';

  arr[1] := 10;
  arr[2] := 20;

  while data.i <= 10 do
    data.i := data.i + 1;

  for data.j := 10 downto 1 do
    data.j := data.j - 1;

  for data.i := 1 to 3 do
    data.i := Hitung(data.i);

  repeat
    data.i := data.i + 1;
  until data.i >= 10;

  if data.i >= 10 then
```

```
    data.status := 'selesai'
  else
    data.status := 'belum';

  hasil := data.i;

  case hasil of
  0:
    writeln('nol');
  1:
    writeln('satu');
  2:
    writeln('dua');
  end;

  TulisPesan(data.status);
end.
```

Keluaran (output)

```
PROGRAMSY()
IDENT(NormalBangetLengkapProgramnya)
SEMICOLON()
TYPESY()
IDENT(AngkaArray)
UNKNOWN()
ARRAYSY()
LBRACK()
INTCON(5)
RBRACK()
OFSY()
IDENT(integer)
SEMICOLON()
IDENT(DataLoop)
UNKNOWN()
RECORDSY()
IDENT(i)
COLON()
IDENT(integer)
SEMICOLON()
IDENT(j)
COLON()
IDENT(integer)
SEMICOLON()
IDENT(status)
COLON()
IDENT(string)
SEMICOLON()
ENDSY()
SEMICOLON()
VARSY()
IDENT(data)
COLON()
IDENT(DataLoop)
SEMICOLON()
```

```
IDENT(arr)
COLON()
IDENT(AngkaArray)
SEMICOLON()
IDENT(hasil)
COLON()
IDENT(integer)
SEMICOLON()
PROCEDURESY()
IDENT(TulisPesan)
LPARENT()
IDENT(teks)
COLON()
IDENT(string)
RPARENT()
SEMICOLON()
BEGINSY()
IDENT(writeln)
LPARENT()
IDENT(teks)
RPARENT()
SEMICOLON()
ENDSY()
SEMICOLON()
FUNCTIONSY()
IDENT(Hitung)
LPARENT()
IDENT(x)
COLON()
IDENT(integer)
RPARENT()
COLON()
IDENT(integer)
SEMICOLON()
BEGINSY()
IDENT(Hitung)
BECOMES()
LPARENT()
IDENT(x)
IDENT(div)
INTCON(3)
RPARENT()
IDENT(MOD)
INTCON(2)
SEMICOLON()
ENDSY()
SEMICOLON()
BEGINSY()
IDENT(data)
PERIOD()
IDENT(i)
BECOMES()
INTCON(0)
SEMICOLON()
IDENT(data)
PERIOD()
```

```
IDENT(j)
BECOMES()
INTCON(0)
SEMICOLON()
IDENT(data)
PERIOD()
IDENT(status)
BECOMES()
STRING('awal')
SEMICOLON()
IDENT(arr)
LBRACK()
INTCON(1)
RBRACK()
BECOMES()
INTCON(10)
SEMICOLON()
IDENT(arr)
LBRACK()
INTCON(2)
RBRACK()
BECOMES()
INTCON(20)
SEMICOLON()
WHILESY()
IDENT(data)
PERIOD()
IDENT(i)
LEQ()
INTCON(10)
DOSY()
IDENT(data)
PERIOD()
IDENT(i)
BECOMES()
IDENT(data)
PERIOD()
IDENT(i)
PLUS()
INTCON(1)
SEMICOLON()
FORSY()
IDENT(data)
PERIOD()
IDENT(j)
BECOMES()
INTCON(10)
DOWNTOSY()
INTCON(1)
DOSY()
IDENT(data)
PERIOD()
IDENT(j)
BECOMES()
IDENT(data)
PERIOD()
```

```
IDENT(j)
MINUS()
INTCON(1)
SEMICOLON()
FORSY()
IDENT(data)
PERIOD()
IDENT(i)
BECOMES()
INTCON(1)
TOSY()
INTCON(3)
DOSY()
IDENT(data)
PERIOD()
IDENT(i)
BECOMES()
IDENT(Hitung)
LPARENT()
IDENT(data)
PERIOD()
IDENT(i)
RPARENT()
SEMICOLON()
REPEATSY()
IDENT(data)
PERIOD()
IDENT(i)
BECOMES()
IDENT(data)
PERIOD()
IDENT(i)
PLUS()
INTCON(1)
SEMICOLON()
UNTILSY()
IDENT(data)
PERIOD()
IDENT(i)
GEQ()
INTCON(10)
SEMICOLON()
IFSY()
IDENT(data)
PERIOD()
IDENT(i)
GEQ()
INTCON(10)
THENSY()
IDENT(data)
PERIOD()
IDENT(status)
BECOMES()
STRING('selesai')
ELSESY()
IDENT(data)
```

```
PERIOD()
IDENT(status)
BECOMES()
STRING('belum')
SEMICOLON()
IDENT(hasil)
BECOMES()
IDENT(data)
PERIOD()
IDENT(i)
SEMICOLON()
CASESY()
IDENT(hasil)
OFSY()
INTCON(0)
COLON()
IDENT(writeln)
LPARENT()
STRING('nol')
RPARENT()
SEMICOLON()
INTCON(1)
COLON()
IDENT(writeln)
LPARENT()
STRING('satu')
RPARENT()
SEMICOLON()
INTCON(2)
COLON()
IDENT(writeln)
LPARENT()
STRING('dua')
RPARENT()
SEMICOLON()
ENDSY()
SEMICOLON()
IDENT(TulisPesan)
LPARENT()
IDENT(data)
PERIOD()
IDENT(status)
RPARENT()
SEMICOLON()
ENDSY()
PERIOD()
```

Masukkan (Input)

```
program Lexer;

var a = 0;
var b = -2;
```

```
var c = 0.03;

Eql =
Lss <
Gtr >
Colon :
Plus+
Minus-
Times  *
RDIV
/[ ] ,. '' ( )
<> < > <= >= == := : =

not
And
orConst
or Consta
const
while
fof
downto to down to
#
{pagar gak bisa}
```

Keluaran (output)

```
PROGRAMSY()
IDENT(Lexer)
SEMICOLON()
VARSY()
IDENT(a)
UNKNOWN()
INTCON(0)
SEMICOLON()
VARSY()
IDENT(b)
UNKNOWN()
MINUS()
INTCON(2)
SEMICOLON()
VARSY()
IDENT(c)
UNKNOWN()
REALCON(0.0)
SEMICOLON()
IDENT(Eql)
UNKNOWN()
IDENT(Lss)
LSS()
IDENT(Gtr)
GTR()
IDENT(Colon)
COLON()
IDENT(Plus)
PLUS()
```

```
IDENT(Minus)
MINUS()
IDENT(Times)
TIMES()
IDENT(RDIV)
RDIV()
LBRACK()
RBRACK()
COMMA()
PERIOD()
STRING('')
LPARENT()
RPARENT()
NEQ()
LSS()
GTR()
LEQ()
GEQ()
EQL()
BECOMES()
COLON()
UNKNOWN()
NOTSY()
ANDSY()
IDENT(orConst)
ORSY()
IDENT(Consta)
CONSTSY()
WHILESY()
IDENT(fof)
DOWNTOSY()
TOSY()
IDENT(down)
TOSY()
UNKNOWN()
COMMENT(pagar gak bisa)
```

Masukkan (Input)

```
Program testing;

const
  PI := 3.14;

type
  Data = record
    x: integer;
    y: integer;
  end;

var
  d: Data;
  hasil: real;
```

```
procedure print;
begin
  writeln('Hai');
end;

function Pembagian(a, b: integer): real;
begin
  Pembagian := a / b;
end;

begin
  d.x := 8;
  d.y := 3;

  hasil := Pembagian(d.x, d.y);

  if d.x <> d.y then
    print;

  case d.x of
    8:
      writeln('delapan');
    3:
      writeln('tiga');
  end;
end.
```

Keluaran (output)

```
PROGRAMSY()
IDENT(testing)
SEMICOLON()
CONSTSY()
IDENT(PI)
BECOMES()
REALCON(3.1)
SEMICOLON()
TYPESY()
IDENT(Data)
UNKNOWN()
RECORDSY()
IDENT(x)
COLON()
IDENT(integer)
SEMICOLON()
IDENT(y)
COLON()
IDENT(integer)
SEMICOLON()
ENDSY()
SEMICOLON()
VARSY()
IDENT(d)
COLON()
```

```
IDENT(Data)
SEMICOLON()
IDENT(hasil)
COLON()
IDENT(real)
SEMICOLON()
PROCEDURESY()
IDENT(print)
SEMICOLON()
BEGINSY()
IDENT(writeln)
LPARENT()
STRING('Hai')
RPARENT()
SEMICOLON()
ENDSY()
SEMICOLON()
FUNCTIONSY()
IDENT(Pembagian)
LPARENT()
IDENT(a)
COMMA()
IDENT(b)
COLON()
IDENT(integer)
RPARENT()
COLON()
IDENT(real)
SEMICOLON()
BEGINSY()
IDENT(Pembagian)
BECOMES()
IDENT(a)
RDIV()
IDENT(b)
SEMICOLON()
ENDSY()
SEMICOLON()
BEGINSY()
IDENT(d)
PERIOD()
IDENT(x)
BECOMES()
INTCON(8)
SEMICOLON()
IDENT(d)
PERIOD()
IDENT(y)
BECOMES()
INTCON(3)
SEMICOLON()
IDENT(hasil)
BECOMES()
IDENT(Pembagian)
LPARENT()
IDENT(d)
```

```
PERIOD()
IDENT(x)
COMMA()
IDENT(d)
PERIOD()
IDENT(y)
RPARENT()
SEMICOLON()
IFSY()
IDENT(d)
PERIOD()
IDENT(x)
NEQ()
IDENT(d)
PERIOD()
IDENT(y)
THENSY()
IDENT(print)
SEMICOLON()
CASESY()
IDENT(d)
PERIOD()
IDENT(x)
OFSY()
INTCON(8)
COLON()
IDENT(writeln)
LPARENT()
STRING('delapan')
RPARENT()
SEMICOLON()
INTCON(3)
COLON()
IDENT(writeln)
LPARENT()
STRING('tiga')
RPARENT()
SEMICOLON()
ENDSY()
SEMICOLON()
ENDSY()
PERIOD()
()
```

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

Pada tugas ini telah berhasil dirancang dan diimplementasikan sebuah lexical analyzer untuk bahasa Arion menggunakan pendekatan Deterministic Finite Automata (DFA). Digunakan dua pendekatan, yaitu berbasis pola lexeme untuk identifier dan keyword, serta berbasis token eksplisit untuk token lain yang memiliki pola terbatas. Implementasi yang dilakukan mampu mengenali berbagai token sesuai spesifikasi.

5.2 Saran

Untuk pengembangan selanjutnya, lexical analyzer dapat ditingkatkan dengan penanganan error yang lebih komprehensif, seperti mendeteksi input yang tidak valid atau tidak lengkap.

LAMPIRAN

Link Repository Github
https://github.com/Faizalfadaa/JKW-Tubes-IF2224-2026

Link Workspace Diagram Transisi DFA
https://drive.google.com/file/d/1REDoDwv5v5aZLMpf8W8GZmxLilVVv6u0/view?usp=sharing

Pembagian Tugas

NIM	Kontribusi
13524007	25%
13524039	25%
13524069	25%
13524097	25%

REFERENSI

https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm